

Lab 02: Containers & Docker Compose

Operating Systems and Distributed Systems

The purpose of this lab is to get you started with another main tool of this course, Docker-compose.

The lab is structured so the student is assisted through the tasks.

Remember to read the exercise and the accompanying text thoroughly before asking for help.

Contents

Exercise 01: Expose Apache	2
Ports	2
Exercise 02: PHP Server	4
Exercise 03: Ubuntu MySQL	5
MySQL Command-Line Interface (CLI)	5
Exercise 04: MySQL Container	7
Exercise 05: Expose Content	8
Docker Compose	8
YAML files	8

Exercise 01: Expose Apache

In this exercise, the student will spin up a container running an Apache server, expose a port and see the resulting website on the host machine.

Ports

A port is a communication endpoint found within the computer. All connections to a computer are routed through a port, where the communication follows the standard defined by the specific protocol (more about protocols in Lab 04). Ports on remote hosts are accessed with `<host ip>:<port>` while ports on your own machine are accessed with `localhost:<port>`

Theoretically, processes can be assigned to any port within the specified range of 0 – 65535. However, the [Internet Assigned Numbers Authority \(IANA\)](#) has a list of pre-assigned ports that are standardised for specific use cases. Port 80, for example, is pre-assigned for HTTP communication usage. This is why we, by default, do not specify a port when accessing a website in a browser, as the browser assumes that the standard port is in use.

Table 1 lists examples of pre-assigned ports for common protocols.

Table 1: List of useful assigned ports.

Port #	Assignment	Description
20 & 21	File Transfer Protocol (FTP)	...
22	Secure Shell (SSH)	Secure logins, file transfers and communication.
23	Telnet	Unencrypted text communication.
25	Simple Mail Transfer Protocol (SMTP)	E-mail routing between mail servers.
80	HyperText Transfer Protocol	Web Pages.
143	Internet Message Access Protocol (IMAP)	Management of e-mail messages.
443	HTTP Secure (HTTPS)	Encrypted Web Pages

In this exercise, an Apache server is set up in a container, displaying a premade site. The Docker image for this exercise, `cocodolan/php:apache`, can be found on DockerHub.

Your task

1. Make a run command that exposes port 7000 on the host machine to container port 80 and runs the `cocodolan/php:apache` image¹.
2. Visit the forwarded port through a browser on the host machine via <http://localhost>.
After visiting localhost, the exercise is complete when some response is printed to the console.

The exercise will return the following message:

```
1 [Thu Jul 04 13:29:48.343170 2019] [mpm_prefork:notice] [pid 1] AH00163: Apache
   /2.4.25 (Debian) PHP/7.2.19 configured -- resuming normal operations
2
3 [Thu Jul 04 13:29:48.343489 2019] [core:notice] [pid 1] AH00094: Command line: '
   apache2 -D FOREGROUND'
4
5 172.17.0.1 - - [04/Jul/2019:13:20:20 +0000] "GET / HTTP/1.1" 200 370 "-" "Mozilla
   /5.0 (Macintosh; Intel Mac OS X 10_13_6) AppleWebKit/605.1.15 (KHTML, like
   Gecko) Version/12.1.1 Safari/605.1.15"
```

¹**NOTE:** For students using Apple's M processors, the platform needs to be specified:

```
docker run --platform linux/amd64 -p 7000:80 cocodolan/php:apache
```

Exercise 02: PHP Server

In this exercise, the student will create a Dockerfile, build an image from that Dockerfile, expose a port and see the resulting website on the host machine. The container should run a PHP server.

The `.php` files for this exercise are in the `./L2E2` folder on ItsLearning.

Your task

1. Create a Dockerfile that:
 - (a) Uses the `php:apache` image.
 - (b) Runs the command `$ docker-php-ext-install mysqli` in the building phase.
 - (c) Copies the included files to the default root folder that Apache2 uses.
2. Build the Dockerfile. Remember to give the build a tag.
3. Run the above-built image. Remember to expose the port.
4. Visit the forwarded port through a browser on your host machine [<http://localhost>].
 - (a) The site's functionality is not working due to no database connection. However, if the site shows something, it works.

The exercise will return the following message:

```
1 [Fri Sep 06 14:36:34.862957 2019] [mpm_prefork:notice] [pid 1] AH00163: Apache
  /2.4.38 (Debian) PHP/7.3.9 configured -- resuming normal operations
2
3 [Fri Sep 06 14:36:34.863025 2019] [core:notice] [pid 1] AH00094: Command line: '
  apache2 -D FOREGROUND'
4
5
6
7
8 172.17.0.1 - - [06/Sep/2019:14:36:55 +0000] "GET / HTTP/1.1" 302 254 "-" "Mozilla
  /5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/76.0.3809.132 Safari/537.36"
9
10 172.17.0.1 - - [06/Sep/2019:14:36:55 +0000] "GET /insert.php HTTP/1.1" 200 473 "-"
   "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML,
   like Gecko) Chrome/76.0.3809.132 Safari/537.36"
```

Exercise 03: Ubuntu MySQL

In this exercise, the student will spin up an Ubuntu container, install MySQL and try out a list of management commands on the server.

A MySQL server is a database that uses SQL to both manage the server and manipulate the data it contains².

Assuming you already have the hang of SQL, as introduced in a previous course, this exercise focuses on the MySQL CLI and commands for making basic configurations.

MySQL Command-Line Interface (CLI)

A running instance of a MySQL server can be interacted with through their CLI, which can be accessed through a shell/terminal. The command `$ mysql -u root - <YOUR-PASSWORD>` can be used to access the running MySQL server through the CLI.

The appearance of `mysql>` in the command bar indicated that the MySQL server has been accessed. Now, SQL can be used to both manage the server and manipulate data.

Hint 1

Remember that for each query in SQL, the line ending `;` has to be included to indicate a complete SQL command.

Table 2 lists an extract of commonly used commands for MySQL server management.

Table 2: List of commonly used SQL commands for managing a SQL server.

Command	Description
<code>SHOW DATABASES;</code>	Shows all databases inside the MySQL server.
<code>USE <DATABASE>;</code>	Making a database usable.
<code>SHOW TABLES;</code>	Shows the tables in the selected database.
<code>HELP;</code>	Listing all available SQL commands.
<code>EXIT;</code>	Exits the MySQL CLI.

Your task

1. Spin up an interactive `ubuntu` container.
2. Install the `mysql-server` package.
3. Start the MySQL service using `$ service mysql start`
4. Use SQL commands to interact with the database.

NOTE: When accessing the database, include the username using the `-u` option.

```
$ mysql -u username
```

- (a) Show all databases.
- (b) Create a database.
- (c) Show all tables.
- (d) Create a table.
- (e) Insert a row to the table.
- (f) Show the table.
- (g) Remove all data from the table.
- (h) Remove the table
- (i) Remove the database.

²**NOTE:** MariaDB is now the SQL server for ubuntu. Therefore, students can decide to use MariaDB instead of MySQL.

The change should be straightforward, just change all MySQL occurrences with MariaDB.

Exercise 04: MySQL Container

In this exercise, the student will create a Dockerfile that initiates a MySQL server with preconfigured settings. From that Dockerfile, a Docker Image containing a ready-to-use MySQL server is built. Lastly, using the built image, a container is spun up.

Information 1

A container created from the official `mysql` image executes all `.sql` files in the `/docker-entrypoint-initdb.d/` directory³.

This means that any creation and manipulation of configurations, databases, tables and so on in the files will be reflected in the created server.

Your task

1. Create a Dockerfile that:
 - (a) Uses the `mysql` image.
 - (b) Insert `MYSQL_ALLOW_EMPTY_PASSWORD yes` as an environment variable inside the container.
 - (c) Copy the `init.sql` file from the resources `/docker-entrypoint-initdb.d/` directory inside the container.
2. Build the Dockerfile. Remember to give the image a tag.
3. Start a container from the above-created image.
4. Open another terminal and use the `$ docker exec` command to access the MySQL container.
5. Access MySQL with the username `root` and no password.
6. Check that the database and table from the `init.sql` file have been created. If they are created, the exercise is complete.

³**NOTE:** The folder `/docker-entrypoint-initdb.d/` is called `L2E2` in the resources.

Exercise 05: Expose Content

In this exercise, the student will create a `docker-compose.yml` file that composes a system that uses the Dockerfiles made in Exercise 02 and Exercise 04.

Docker Compose

Docker Compose is a tool made for defining and running a multi-container system. This is done using a YAML file `[.yaml]` called a compose file, where the containers and the environment, such as services, networks and volumes, are configured.

In this exercise, the focus will be solely on services, which define configurations for containers.

Using Docker Compose is usually divided into three steps:

1. Define a container environment in a Dockerfile that does some individual work just like we already have been doing.
2. Define services for each container in a compose file so the multi-container system can be run in an isolated but interconnected environment.
3. Run the YAML file using `$ docker compose up` while inside the folder with the YAML file.

Information 2

No context is given when starting up Docker Compose.

YAML files

A YAML file is a general file format often used for configuration files. In Docker Compose, the file is used to configure services to run them together.

A specific syntax is used inside the YAML file to create and describe the environment spun up with the containers.

For starters, there are two things to notice inside a YAML file. Firstly, that `#` used at the beginning of a line is treated as a comment. This means that the Docker engine skips over the lines beginning with a `#`.

Secondly, instructions in a YAML file follow the syntax `<KEY>: <VALUE>`.

```
1 # A docker compose YAML file always starts by defining the version of docker
   compose that is used.
2 # In this case, version 3 is used.
3 version: "3"
```



```

4
5 # Using the key services, we indicate that the following lines will define the
   services included in the environment.
6 services:
7
8 # After the services key, the services are defined. Here, you include a key holding
   the name of your service.
9 # In this case, we have named the service "sql as it describes the role of the
   service.
10 sql:
11
12 # After a service has been made, several options are available to specify what the
   service should be built from and which properties the service should have.
13 # Build is used to specify the file path to where the Dockerfile for the SQL
   service is located. The path should be relative to the docker compose.
14     build: <PATH-TO-SQL-DOCKERFILE>
15
16     web:
17         build: <PATH-TO-WEB-DOCKERFILE>
18
19 # Depends_on tells Docker that the web service depends on the sql service. This way
   , docker creates a dependency graph where the containers without dependencies
   are first started.
20     depends_on:
21     - sql
22
23 Ports work like -p 8080:50, making the container's port 80 accessible on the host
   port 5050.
24     ports:
25     - "5050:80"

```

Information 3

Due to increasing security measures in MySQL, connecting a service using a regular password is disabled by default. To circumvent this, add `--default-authentication-plugin=mysql_native_password` as a command in the sql service.

Information 4

Ensure the database service's name matches the hostname defined in the `config.php` in the web service directory.

Your task

1. Create a `docker-compose.yml` that has:
 - (a) A db service
 - i. That builds from a `sql`-subfolder where the `sql` Dockerfile is located.
 - (b) A web service
 - i. That depends on the db service
 - ii. That is built from a `web` subfolder where the `web` Dockerfile is located.
 - iii. That exposes port 80 to the host's port 7000.
2. Run the docker compose
3. Check <http://localhost> now, and try to insert some data!

The exercise will return the following message:

```
1 [Thu Jul 04 13:29:48.343170 2019] [mpm_prefork:notice] [pid 1] AH00163: Apache
  /2.4.25 (Debian) PHP/7.2.19 configured -- resuming normal operations
2
3 [Thu Jul 04 13:29:48.343489 2019] [core:notice] [pid 1] AH00094: Command line: '
  apache2 -D FOREGROUND'
4
5
6
7 172.17.0.1 - - [04/Jul/2019:13:20:20 +0000] "GET / HTTP/1.1" 200 370 "-" "Mozilla
  /5.0 (Macintosh; Intel Mac OS X 10_13_6) AppleWebKit/605.1.15 (KHTML, like
  Gecko) Version/12.1.1 Safari/605.1.15"
```